

TM1 COPILOT AI

PA-Copilot

Capabilities Reference & Feature Documentation

The complete guide to the AI copilot for IBM Planning Analytics.
How every feature works, why it matters, what to expect, with real examples,
use cases, limitations and best practices — grounded in live testing.

AI ENGINE

Anthropic Claude (via API)

STACK

FastAPI · Next.js · TM1py · pgvector

VERIFIED ON

IBM PAW Trial · 24 cubes · 61 dimensions · 27 processes

Contents

1. Introduction — what PA-Copilot is and who it's for
2. Architecture and how it works
3. The request lifecycle (verification pipeline)
4. AI capabilities in depth
5. The nine specialist agents
6. The human-gated deployment engine
7. TM1 code generation — correctness and coverage
8. Security, roles and governance
9. Monitoring and cost governance
10. Getting the most accurate results
11. Limitations and known behaviour
12. Best-practice checklist
13. Glossary

1

Introduction — what PA-Copilot is and who it's for

PA-Copilot is an AI engineering assistant built specifically for IBM Planning Analytics (TM1). Unlike a general coding assistant, every capability is scoped to real TM1 work and grounded in the actual model it is connected to. It writes and explains TurboIntegrator (TI) processes, reasons about rules and feeders, turns plain-language questions into MDX and charts, explains errors, and safely drafts changes to the model — where a human reviews and applies every write.

The design goal is narrow and deliberate: reduce the time spent writing and debugging routine TM1 work, while ensuring every change to a production model remains under human control.

Who it is for

Audience	What they get
Developers	Grounded, compile-checked TI, rules and MDX; fast explanation of unfamiliar objects.
Administrators	Impact-aware investigation of chores and processes before acting.
Consultants	Rapid model documentation and design review against good practice.
Business users	Answers and charts from plain-English questions, with no MDX required.

The problem it solves

General AI tools generate TM1 code that looks correct but references objects that do not exist — discovered only when it reaches the server. PA-Copilot reads the live model first and validates against it, so confident-but-wrong output is caught before it costs you time.

PA-Copilot is a standalone web application with a clear separation between the interface, the reasoning layer, and the connection to TM1. It is not a front end for any third-party copilot product; the platform is custom-built, and the language model is consumed as a metered API service.

Layer	Technology	Role
Frontend	Next.js (React/TypeScript)	The console: chat, visualize, deployments; streaming responses, voice, light/dark.
Backend	FastAPI (Python, async)	Agent orchestration, per-tool permissions, usage and cost accounting.
TM1 access	TM1py over the TM1 REST API	All reads and writes to TM1; circuit-breaker resilience; live compile validation.
Reasoning	Anthropic Claude (via API)	The language model behind every agent.
Knowledge	PostgreSQL + pgvector	Stores the metadata graph, change history, and Knowledge Base embeddings.

Grounding: the core idea

Every specialist agent is given a scoped set of tools and is instructed never to use a cube, dimension or element name it has not confirmed with a tool call. In practice the agent first calls read tools (for example `list_cubes`, `get_dimension`, `list_dimension_elements`) against the live model, then reasons and writes using only the names those tools returned. This is what prevents the most common AI failure — inventing plausible names that do not exist.

Standards grounding

The code-generating agents are additionally required to search the organization's Knowledge Base before drafting any process, rule or feeder. Uploaded reference TI, rules and standards documents define how the organization writes code — naming conventions, structure, logging, error handling and comment style — and generated code follows those conventions, citing which standards were applied. Where no relevant standard exists, the agent says so plainly and applies mainstream TM1 practice instead of presenting an invented convention as the organization's own.

Metadata graph

A one-time 'Extract metadata' step reads the whole model and builds a dependency graph in the application's database — objects as nodes, 'uses dimension' and 'references cube' as edges (the latter parsed from rule text). Relationship views, the interactive graph, and the impact analysis attached to every drafted change all read from this graph. It is rebuilt from scratch each time extraction runs and never writes to TM1.

The request lifecycle (verification pipeline)

PA-Copilot does not generate code and hope for the best. Every substantive request follows the same governed path.

1. Inspect the live model

The agent reads the real cubes, dimensions and elements involved before writing anything.

2. Retrieve organizational standards

Relevant TI, rules and standards are pulled from the Knowledge Base as context — a mandatory step for code generation.

3. Generate context-aware output

Code or a query is written against the model and the organization's conventions, not a generic template.

4. Verify against the server

TurboIntegrator drafts are compiled on the live TM1 server — without being saved — so syntax errors are caught and revised before anyone sees them.

5. Present a reviewable diff

For changes, the console shows current vs proposed, with impact analysis from the graph.

6. Snapshot before execution

The previous version is captured so any change can be undone.

7. Auto-roll-back on failure

If post-write verification fails, the previous version is restored automatically.

What this guarantees

No blind execution, no silent changes. The AI can produce only drafts; the single action that writes to TM1 is a permitted human clicking Execute on a reviewed change.

Each capability below follows the same structure: what it is, why it matters, how it works, expected input and output, an example, use cases, limitations and best practices.

4.1 AI Chat and the agent model

What it is. The central workspace where you converse with nine specialist agents that can use live TM1 tools.

Why it matters. It concentrates all AI work in one place with full transparency: you always see which agent acted, which TM1 tools it called, and which objects it touched.

How it works (high level). You pick an agent, which enables that agent's specific tool set. Your message and recent history go to the language model with a system prompt and the tool definitions; the model calls tools in a loop (bounded per agent) until it can answer, streaming the reply back over Server-Sent Events. Plain chat (no agent selected) has no tools, and is instructed to direct code-generation requests to the TI or Developer agents rather than produce ungrounded code.

Expected input. A plain-language message; optionally a selected agent, an attached PDF/JPG/PNG/DOCX, or voice dictation.

Expected output. A streamed answer, a Tool Execution Timeline listing every TM1 call made, and a Referenced Objects list. Conversations are saved, searchable, renamable and deletable.

Example

Prompt (TI agent): "Create a TI process to build the Product dimension from a CSV."

Response: the agent searches the Knowledge Base for standards, checks existing processes and the target structure, drafts the process (compile-validated live), then explains it section by section and lists its assumptions.

Real-world use cases

- Draft and debug TI without leaving the model context.
- Explain an unfamiliar process to a new team member.
- Investigate why a chore behaves unexpectedly.
- Review a cube's rules for issues before a release.

Limitations

- Answer quality depends on the metadata the chosen agent can read.
- Voice input uses the browser's speech engine (Chrome/Edge recommended).
- Each agent has a bounded number of tool rounds, so extremely broad requests may need to be narrowed.

Best practices

- Pick the agent that matches the task.
- Name real cubes, periods and measures.
- Read the Tool Execution Timeline to confirm grounding before trusting output.

4.2 TI Script Assistant

What it is. Generation and updating of TurboIntegrator processes, drafted as reviewable changes and validated against TM1.

Why it matters. TI is where most repetitive TM1 development time goes, and where confident-but-wrong AI code is most costly. Live compile validation and standards grounding remove that risk.

How it works (high level). Before drafting, the TI agent is required to search the organization's Knowledge Base for relevant coding standards and reference processes, so generated code follows the organization's own conventions (naming, structure, logging, error handling, comment style) rather than generic patterns — and it cites which standards it applied. It then reads similar existing processes and the target cube/dimension structure, builds the process via a proposal tool (constructed with TM1py, so the Prolog/Metadata/Data/Epilog structure is valid by construction), and the draft is compiled on the live server without being saved. Any syntax errors are returned and the agent revises before you see the draft. File-driven loaders can declare an ASCII datasource, source-column variables and process parameters so they compile correctly.

Expected input. A description of the process you want, ideally naming the real objects and any parameters.

Expected output. A drafted process with all four sections, a validation status, an impact estimate, and a plain-language explanation of what each section does plus anything to double-check.

Example

Verified example (generated live, compiled cleanly, zero validation errors):

Real-world use cases

- Build or refresh dimensions from a source file.
- Bulk element or attribute maintenance.
- One-off data fixes wrapped as a safe, re-runnable process.
- Refactor an existing process to a cleaner pattern.

Limitations

- Compile validation catches syntax, not business logic — human review still matters.
- The agent drafts; it cannot execute. A human with deploy rights applies the change.

Best practices

- State parameters explicitly in your request.
- Review the listed assumptions before executing.
- Prefer small, single-purpose processes that are easy to review.

```
# Create the dimension if it does not exist, then rebuild it idempotently
IF( DimensionExists( sDim ) = 0 ); DimensionCreate( sDim ); ENDIF;
DimensionDeleteAllElements( sDim );
DimensionElementInsert( sDim, '', 'All Products', 'C' );
DimensionElementInsert( sDim, '', 'Laptop', 'N' );
DimensionElementComponentAdd( sDim, 'All Products', 'Hardware', 1 );
```

Every function signature, element type code (C/N) and statement order above is valid TM1, and the draft compiled on the live server on the first attempt.

4.3 Rules & Feeders Intelligence

What it is. Plain-language explanation and drafting of cube rules and feeders.

Why it matters. Rules and feeders are the hardest TM1 code to reason about and the riskiest to change; clear explanation and a safe apply path reduce that risk.

How it works (high level). The Developer agent searches the Knowledge Base for the organization's rule and feeder standards, reads the cube's rules and traces read/write dependencies from the metadata graph. Rule changes are drafted for review. On execution the engine applies the rule, re-checks it on the server, and automatically restores the previous rules if the check fails.

Expected input. The cube name, and a description of the rule/feeder behaviour you want or the rule you want explained.

Expected output. A plain-language explanation grounded in the real objects, or a drafted rule change with a diff and impact.

Real-world use cases

- Understand an inherited rule before modifying it.
- Draft a phasing or allocation rule for review.
- Trace what a feeder is doing before changing performance behaviour.

Limitations

- TM1 has no API to compile a rule without saving it, so rule drafts are validated at execution time (apply, check, restore-on-failure) rather than beforehand — a rule draft can look clean yet fail on execute, and is then auto-restored.
- Feeders follow the same path as rules.

Best practices

- Change rules in small, reviewable steps.
- Keep a note of expected behaviour so verification is meaningful.
- Use the metadata graph to check dependents before applying.

4.4 Visualize

What it is. Natural-language data questions turned into an MDX query and a live chart.

Why it matters. It removes the MDX skill barrier so business users can self-serve answers, while developers get a correct query they can reuse.

How it works (high level). The Analyst agent finds the right cube, confirms real element names, writes MDX and runs it to prove it works. The final query is then re-executed to return the full cell set behind the chart.

Expected input. A specific question naming the cube, measure and period; optional chart type and size.

Expected output. A one-line summary, the exact MDX used, and a chart (bar/line/area). Results are capped at 500 cells.

Example

Prompt: "Show me total revenue by month from the Sales cube."

Verified response — the agent produced correct MDX and used the model's irregular element name [P11Y] (no space) rather than guessing [P11 Y]:

Real-world use cases

- Quick monthly/quarterly figures for a review.
- Sanity-check a number without opening a view.
- Generate a starting MDX query to adapt in your own reports.

Limitations

- Results are capped at 500 cells — narrow very large questions.
- Element names must exist; vague questions get vaguer answers.

Best practices

- Name the cube, measure and period explicitly.
- Reuse the emitted MDX where helpful.
- Narrow the period or entity for a cleaner chart.


```
SELECT {[Time].[P1 Y], ... , [Time].[P11Y], [Time].[P12 Y]} ON COLUMNS,
       {[Sales].[Revenue]} ON ROWS
FROM [Sales]
WHERE ([Scenario].[Budget], [Channel].[Combined], [Geography].[All Regions], [Product].[Product
      Total])
```

4.5 Knowledge Base & Explain Error

What it is. Grounding answers and generated code in the organization's own uploaded documentation (retrieval-augmented generation), plus a dedicated error explainer.

Why it matters. It captures and reuses the organization's standards so answers and generated code match how the team works, and turns cryptic TM1 errors into actionable fixes.

How it works (high level). Uploaded documents are chunked and embedded into a vector store; relevant excerpts are retrieved as context when you ask. The code-generating agents (TI and Developer) search this same library automatically before drafting, so uploaded TI, rule and naming standards directly shape generated code. Explain Error routes to the Troubleshooter agent, which forms a hypothesis and checks the real objects the error mentions.

Expected input. Knowledge Base: documents (PDF/DOCX/TXT/MD) then a question. Explain Error: the raw error text.

Expected output. Knowledge Base: an answer grounded in your library, with source citations, and an optional 'documents only' mode. Explain Error: likely cause, step-by-step remediation, and what to check next.

Example

Explain Error input: a pasted "Invalid key" metadata error.

Response: a structured diagnosis — likely cause (a write-before-create ordering issue), remediation steps, and follow-up checks.

Real-world use cases

- Answer 'what is our naming standard?' from your own docs.
- Have generated TI/rules follow the team's uploaded reference code.
- Onboard new developers with self-serve answers.
- Diagnose a logon, feeder or recursion error quickly.

Limitations

- Retrieval quality depends on what you upload; keep documents current.
- The assistant cannot read TM1 security groups, so it will not diagnose access-group errors — it says so rather than guessing.

Best practices

- Upload naming standards, reference TI and model docs early.
- Use 'documents only' when you need answers strictly from your library.
- Paste the exact error text, not a paraphrase.

4.6 Metadata Explorer & dependency intelligence

What it is. A browsable map of the model and an interactive dependency graph, powering impact analysis.

Why it matters. It answers the hardest question in TM1 development — 'if I change this, what breaks?' — before you make the change.

How it works (high level). After metadata extraction, the explorer reads the dependency graph to show 'depends on' and 'depended on by' relationships and an interactive node graph. The same graph feeds the impact list attached to every drafted change.

Expected input. A selected connection and an object to inspect.

Expected output. Overview (structure and, for cubes, rule text), Relationships, an interactive Graph, Recent Changes, and an 'Ask the AI' shortcut.

Example

Verified example: for the BalanceSheet cube the graph resolves 12 connected nodes — six dimensions it uses and five cubes it references, with cube-to-cube links parsed from the rule text (for instance a DB() reference to IncomeStatement).

Real-world use cases

- Assess blast radius before a rule or process change.
- Document a model you have inherited.
- Find unused objects as refactoring candidates.

Limitations

- Reflects the last extraction — re-run it after model changes for accurate impact.
- Requires the one-time metadata extraction per connection.

Best practices

- Extract metadata after any significant model change.
- Check 'depended on by' before editing shared objects.

The nine specialist agents

PA-Copilot routes work to a specialist persona, each with its own scoped tools and guardrails. All are metadata-grounded and honest about their limits; only two can draft changes, and every non-code agent directs code-generation requests to them.

Agent	Role	Example request	Can draft?
TI	Generate/update TI processes	'Create a TI process to load Products from CSV'	Yes — TI
Developer	Explain TI, rules, cube structure	'Explain what the Update Period process does'	Yes — rules/TI
Analyst	NL data questions → MDX → data	'What was cash flow last quarter?'	No
Architect	Design, naming, dependency review	'Review the Sales cube design'	No
Administrator	Chore/process operations	'Why did the nightly chore fail?'	No
Reviewer	Severity-ranked object review	'Review these rules against good practice'	No
Performance	Complexity & fan-out analysis	'Which cubes have the heaviest rules?'	No
Troubleshooter	Explain errors + remediation	'Explain this feeder error'	No
Documentation	Generate model documentation	'Document the BalanceSheet cube'	No

Each agent's system prompt fixes its behaviour: what it may do, which tools it may call, and what it must not claim. For example, the TI agent must consult the Knowledge Base and report validation errors rather than leave a broken draft; the Analyst must confirm real element names before writing MDX; every agent must say it cannot access TM1 security groups rather than guessing. Because personas are configuration, new specialists can be added without changing application code.

The human-gated deployment engine

How a drafted change becomes a safe, reversible write to TM1.

Every change moves through an explicit lifecycle, and each transition is recorded for audit.

Stage	What happens
Draft	The agent proposes the change. TI drafts are compile-validated live; impact is computed from the graph. The draft is stored with any validation errors.
Review	A permitted human opens the change and sees current vs proposed content, the impact list, and the validation status.
Execute	Blocked if validation errors exist. The previous version is snapshotted, the change applied, then re-verified on the server.
Auto-restore	If post-write verification fails, the previous version is restored immediately and the change is marked failed.
Reject	A draft can be discarded without ever touching TM1.
Roll back	An executed change can be reversed later by restoring its snapshot.

Where snapshots live

The 'previous version' used for rollback is stored in the application's own database, not in TM1 (TM1 has no object-level sandbox). Protect that database with backups — it is the safety net behind every reversible change.

The correctness of generated code rests on several reinforcing mechanisms, not on trusting the model.

- **Grounded in the model** — reads real cube/dimension/element names before generating.
- **Grounded in standards** — searches the organization's Knowledge Base before drafting and follows the conventions it finds, citing them.
- **Structurally valid** — processes are built via TM1py, so their section structure is correct by construction.
- **Live compile** — every TI draft is compiled on the server before you see it; syntax errors are returned and revised.
- **Human-gated execute** — blocked if validation errors exist; snapshot, apply, verify, auto-restore on failure.
- **Single-attempt writes** — a failed write is never blindly retried.
- **Professional output** — comments on their own lines only, no placeholder text, no AI attribution; drafts read as if written by an experienced TM1 developer.
- **Scoped and audited** — a write permission is required and every draft and execution is logged.

Coverage by task

TM1 task	How it is handled	Confidence
Dimension / hierarchy / attribute	TI drafts (create/refresh), compile-validated	High
Dynamic subset (MDX)	MDX-based subset in TI; real names confirmed	High
Static subset	TI subset creation + element adds, compile-validated	High
TurboIntegrator processes	Core path: grounded, drafted, compile-validated	High
File-driven loaders	ASCII datasource, source columns and parameters declared	High
MDX queries	Written and executed live (verified accurate)	High
Cube / view creation	TI-driven, grounded in the existing model	Med-High
Rules	Drafted and explained; validated at execute (no dry-run)	Medium
Feeders	Same path as rules — validate by executing	Medium
Chores	Read / investigate / document	Med (read)
Security processes	Not covered — no access to security groups	N/A (by design)

Confidence reflects how strongly the pipeline can guarantee correctness: High means live compile or execution validation; Medium means safe apply-with-restore but no pre-validation.

Access control and auditability are how the platform works, not add-ons.

Role-based access

Role	Typical capability
Super Admin	Full platform administration.
Organization Admin	Manage users, roles, connections, deployments.
Planner	Use AI and create drafts.
Analyst	Query and visualize; read-only on the model.
Viewer	Read-only.

Controls

- **No unattended writes** — AI tools create drafts only; execute/rollback require a permitted human, enforced in code.
- **Per-tool permissions** — each tool checks a specific permission (read / write / chat) before running.
- **Encrypted credentials** — TM1 connection secrets are encrypted at rest with a dedicated key.
- **Least privilege** — the AI never needs TM1 user passwords or security-group membership to generate code.
- **Admin-approved onboarding** — new users request access and an administrator approves before sign-in.
- **Full audit trail** — every drafted change and tool execution is recorded with who and when.
- **Resilience** — a per-connection circuit breaker isolates a failing server instead of cascading failures.

AI usage and platform health are visible and bounded.

- **Usage by model** — requests and tokens per AI model.
- **Tool execution audit** — per-tool call counts, successes, errors and average duration; the first place to look if the AI misbehaves.
- **Circuit-breaker state** — live health and failure count per connection.
- **Enforced quota** — a monthly token limit caps runaway use; requests are rejected once the cap is reached.

Reading the tiles

On the Dashboard and Monitoring, a circuit-breaker state of 'closed' means healthy (traffic flows). 'open' means the breaker has tripped after repeated failures and is protecting the app.

- Pick the agent that matches the task (TI to write, Analyst for data, Troubleshooter for errors).
- Name real objects — cube, dimension, measure and period. Specific questions get grounded, correct answers.
- Give one clear goal per message; refine with follow-ups rather than stacking requests.
- Attach context — a spec PDF, an error screenshot, or a Word doc — so the agent reasons over it.
- Upload reference TI scripts, rules and naming standards to the Knowledge Base — the code agents consult them automatically, so the more context they have, the closer output matches your conventions.
- Read the Tool Execution Timeline to confirm the agent grounded itself before trusting output.
- Narrow large data questions to stay under the 500-cell cap.
- Always review drafts in Deployments — especially rules and feeders, which are validated only at execute.

Area	What to know
Rule / feeder validation	No TM1 rule dry-run exists, so rule drafts are validated at execute time (then auto-restored on failure).
Compile scope	Compile validation catches syntax, not business logic — human review still matters.
Query size	Data questions are capped at 500 cells; narrow the period, account or entity.
Security data	The AI cannot read TM1 security groups or user data, and says so rather than guessing.
First-load latency	The hosted environment sleeps when idle and may take up to a minute to wake.
Response time	Answers typically take 30–60 seconds because the model reads the live model first.
Rollback storage	Snapshots live in the application database, not TM1 — protect it with backups.

Before you generate

- Confirm the right connection is selected and its breaker is closed.
- Choose the agent that matches the task.
- Name the real cube, dimension and period in your prompt.

Before you execute a change

- Read the current vs proposed diff.
- Check the impact list — are the dependents expected?
- Confirm validation status; remember rules are only validated at execute.
- Prefer small, reviewable changes.

Ongoing

- Keep the Knowledge Base current — it directly shapes generated code.
- Re-run metadata extraction after model changes.
- Watch usage and connection health in Monitoring.
- Grant least-privilege roles and revoke promptly.
- Back up the application database — it holds change history and rollback snapshots.

Term	Meaning
TI (TurboIntegrator)	TM1's scripting/ETL engine; processes have Prolog/Metadata/Data/Epilog sections.
Rule / Feeder	Cube calculation logic; feeders tell TM1 where calculated values may exist (for performance).
MDX	The query language used to read cube cell data.
Subset	A named selection of dimension elements; dynamic (MDX-driven) or static (fixed list).
Chore	A scheduled sequence of TI processes.
Draft change	An AI-proposed change not yet applied; requires human execution.
Compile (dry-run)	Validating a TI process against the live server without saving it.
Metadata graph	The dependency map the app builds from the model to power relationship and impact analysis.
Circuit breaker	A resilience mechanism that trips when a connection repeatedly fails, protecting the app.
RAG	Retrieval-augmented generation — grounding answers in your uploaded documents.
RBAC	Role-based access control.
Grounding	Reading the real model and standards before generating, so output uses names and conventions that actually exist.

This document reflects PA-Copilot as evaluated through live testing on an IBM Planning Analytics Workspace trial environment. Features and limits may change between releases; confirm behaviour in your environment.